

# Performance Analysis of Pathfinding in Unity

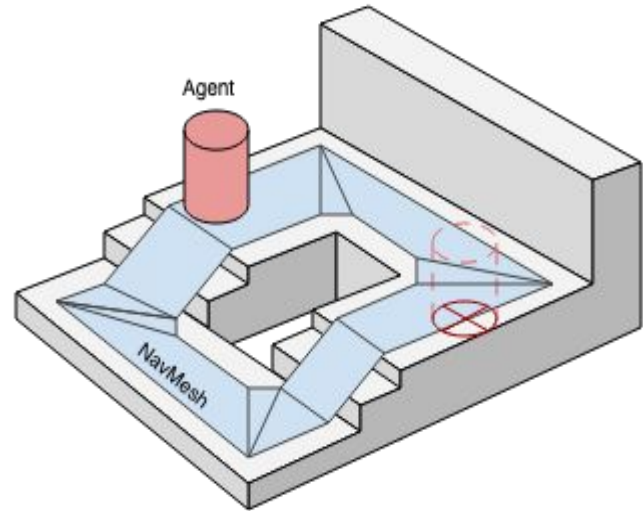
Eray Eroğlu 2020400096

# Problem Definition

- Pathfinding is a plotting node to find the shortest or minimum path between two points, which is from source to destination by a computer application
- In game development, pathfinding algorithms are at the core of creating realistic and intelligent behaviors for NPCs in video games
- Finding optimal paths for multiple agents in real-time is a major challenge in game AI
- Unity has a AI navigation package named NavMesh

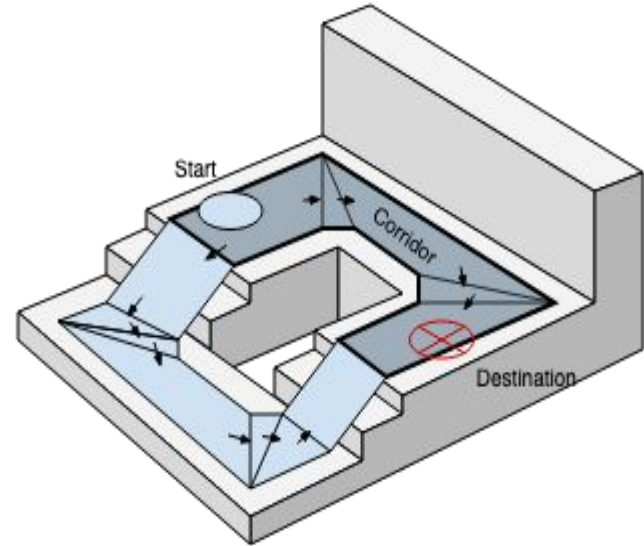
# Unity Navmesh

- The navigation system needs its own data to represent the walkable areas in a game scene. The walkable areas define the places in the scene where the agent can stand and move.
- In Unity the agents are described as cylinders. The walkable area is built automatically from the geometry in the scene by testing the locations where the agent can stand
- Then the locations are connected to a surface laying on top of the scene geometry. This surface is called the navigation mesh (NavMesh for short)



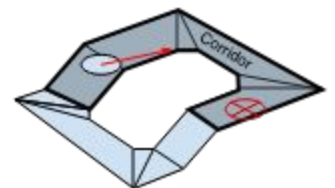
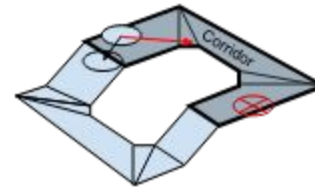
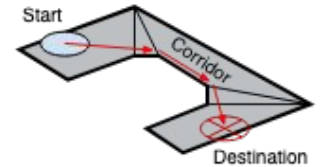
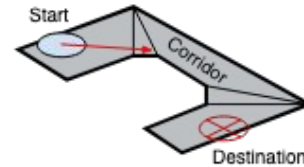
# Finding Paths

- To find a path between two locations in the scene, we first need to map the start and destination locations to their nearest polygons
- Then we start searching from the start location, visiting all the neighbors until we reach the destination polygon.
- Tracing the visited polygons allows us to find the sequence of polygons which will lead from the start to the destination



# Following the Path

- The sequence of polygons which describe the path from the start to the destination polygon is called a corridor.
- The agent will reach the destination by always steering towards the next visible corner of the corridor.
- If you have a simple game where only one agent moves in the scene, it is fine to find all the corners of the corridor in one swoop and animate the character to move along the line segments connecting the corners.
- When dealing with multiple agents moving at the same time, they will need to deviate from the original path when avoiding each other.





From **Sid Meier's Civilization V**



From **Starcraft**



From **Dark Soul's 1**



# Experiment Design & Goal

- The goal of experiment is to evaluate and compare the **performance scalability** of Unity's NavMesh algorithm under increasing computational load
- Wanted to observe how the system behaves when
  - The number of agent increases
  - Path updates happened at increased frequencies
  - The environment contains dynamic obstacles
- This aims to simulate **real-time, dynamic gameplay scenarios** where efficient pathfinding is critical to avoid frame drops and unresponsive AI

# Experiment Setup

- A flat terrain (Plane) acts as the environment.
- Agents are instantiated from a prefab and navigate using either NavMesh.
- Tests are run **in controlled scenes** where the **only changing variable is one of the selected parameters** (agent count, update rate, or obstacle density), ensuring fair comparison.
- For dynamic obstacles, cubes are spawned and repositioned periodically to simulate a changing environment.

# Independent Variables

| Parameter        | Values                 | Why It Was Included                            |
|------------------|------------------------|------------------------------------------------|
| Agent Count      | 1, 4, 8                | Tests scalability under load                   |
| Path Update Rate | Every 1, 5, 10 seconds | Simulates how the algorithm reacts changes     |
| Obstacle Count   | 4, 8, 16               | Tests the algorithm in more dense environments |

# Controlled Variables

| Parameter     | Value       | Why Fixed                                                 |
|---------------|-------------|-----------------------------------------------------------|
| Grid size     | 20x20       | Avoid grid size bias                                      |
| Agent speed   | 2.5 units/s | Ensures movement speed doesn't affect the decision making |
| Test duration | 25 seconds  | Enough time to gather stable performance metrics          |

# Expected Outcomes

| Parameter             | Expected Behaviour                                                                       |
|-----------------------|------------------------------------------------------------------------------------------|
| Agent Count           | Linearly increases CPU load                                                              |
| Memory Usage          | Will be relatively stable, tied more to NavMesh size than agent count                    |
| FPS                   | Will remain stable up to a point, then decline sharply beyond a certain number of agents |
| Dynamic obstacles     | Causes CPU spikes when the obstacles moves                                               |
| Path Update Frequency | Similar to the previous one, causes CPU spikes                                           |

# References

- [Unity documentation](#)
- [Gameplay videos](#)